

COMPTE RENDU

Programmation Système — Projet N°2 — Processus en Anneau
3^e année Cybersécurité — École Supérieure d'Informatique et du
Numérique (ESIN)
Collège d'Ingénierie & d'Architecture (CIA)

Étudiant : HATHOUTI Mohammed Taha
Filière : Cybersécurité
Année : 2025/2026
Enseignants : M. Noufel & M. ElAmrani
Date : 16 mars 2026

Remerciements

Ce projet a été réalisé dans le cadre du cours de Programmation Système dispensé par **M. Noufel** et **M. ElAmrani** à l'UIR durant l'année universitaire 2025-2026.

Je tiens à remercier les enseignants pour la qualité de leur enseignement, et notamment pour les travaux pratiques (TP0 à TP5) qui ont constitué les bases nécessaires à ce projet : gestion des processus, communication inter-processus par tubes (pipes), ordonnancement, synchronisation et signaux sous Linux.

Je souhaite également remercier les enseignants des formations antérieures :

- Les professeurs de l'**Université Grenoble Alpes** (INF301, INF304, INF401) pour la rigueur en programmation et algorithmique ;
- Les professeurs de **Polytech Grenoble** (PROG C, CPS, ALG, ALM) pour la maîtrise du langage C et de la programmation système.

Mohammed Taha HATHOUTI
Mars 2026

Table des matières

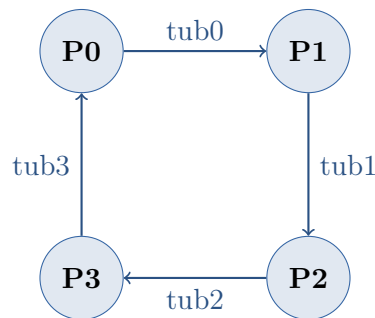
1	Introduction	3
1.1	Contexte	3
1.2	Rappel de la consigne	3
1.3	Structure des fichiers	3
2	Partie a — Création de l’anneau	4
2.1	Principe	4
2.2	Fermeture des descripteurs	4
2.3	Utilitaires de synchronisation	5
3	Partie b — Comptage des processus	5
3.1	Principe	5
3.2	Résultats	6
4	Partie c — Circulation d’un message	6
4.1	Principe	6
4.2	Point critique — retour du 0 final	7
5	Partie d — Circulation des PIDs et somme	7
5.1	Principe	7
5.2	Vérification des résultats	8
6	Partie e — Élection (algorithme de Chang-Roberts)	8
6.1	Principe de l’algorithme	8
6.2	Terminaison	9
6.3	Résultats	9
7	Analyse du bug principal et correction	10
7.1	Symptôme observé	10
7.2	Cause racine	10
7.3	Correction appliquée	10
8	Synthèse et conclusion	11
8.1	Bilan des résultats	11
8.2	Conclusion	11

1 Introduction

1.1 Contexte

Un réseau en anneau (*ring network*) est une topologie classique de communication inter-processus en programmation système. n processus sont organisés en cercle et chacun communique avec ses deux voisins via des tubes Unix (`pipe`). Ce modèle est largement utilisé dans les algorithmes distribués : élection de coordinateur, diffusion d'informations, barrières de synchronisation.

La figure ci-dessous illustre la topologie pour $n = 4$ processus :



Chaque processus i lit depuis `tubes[i][0]` et écrit dans `tubes[(i+1)%n][1]`. Le flux est unidirectionnel et orienté dans le sens $P_0 \rightarrow P_1 \rightarrow \dots \rightarrow P_{n-1} \rightarrow P_0$.

1.2 Rappel de la consigne

Le projet consiste à implémenter cinq fonctionnalités progressives sur cet anneau :

- **a.** Créer les n processus et mettre en place la topologie en anneau via n pipes.
- **b.** Faire compter le nombre de processus par P0 (initiateur).
- **c.** Faire circuler un message entier initialisé à 10, décrémenté par P0 à chaque tour.
- **d.** Faire circuler les PIDs et afficher leur somme.
- **e.** Implémenter l'algorithme d'élection de Chang-Roberts pour élire le processus ayant le plus grand PID.

1.3 Structure des fichiers

Fichier	Rôle
<code>anneau.h</code>	Déclarations, includes, constante <code>MAX_PROCS = 20</code>
<code>anneau.c</code>	Implémentation de toutes les fonctions
<code>main.c</code>	Orchestration des parties b, c, d, e
<code>Makefile</code>	Compilation modulaire avec nettoyage automatique

2 Partie a — Création de l'anneau

2.1 Principe

Les n tubes sont créés **avant** tout `fork()` afin que tous les processus héritent de l'ensemble des descripteurs. Ensuite, $n - 1$ `fork()` créent les processus fils. Chaque fils sort de la boucle immédiatement via `break` pour ne pas créer de sous-fils indésirables.

Listing 1 – Création de l'anneau — `creer_anneau()`

```
1 int creer_anneau(int n, int tubes[][2]) {
2     /* Ouverture des n tubes avant tout fork */
3     for (int i = 0; i < n; i++) {
4         if (pipe(tubes[i]) == -1) { perror("pipe"); exit(1); }
5     }
6
7     int rang = 0;
8     for (int i = 1; i < n; i++) {
9         pid_t pid = fork();
10        if (pid < 0) { perror("fork"); exit(1); }
11        if (pid == 0) { rang = i; break; } /* le fils sort */
12    }
13
14    /* Fermeture selective des descripteurs inutiles */
15    for (int i = 0; i < n; i++) {
16        if (i != rang && i != (rang + 1) % n) {
17            close(tubes[i][0]); close(tubes[i][1]);
18        } else if (i == rang) {
19            close(tubes[i][1]);
20        } else if (i == (rang + 1) % n) {
21            close(tubes[i][0]);
22        }
23    }
24    return rang;
25 }
```

2.2 Fermeture des descripteurs

Pourquoi fermer les descripteurs inutiles ?

Chaque processus hérite de **tous** les descripteurs après `fork()`. Si un processus conserve le bout écriture d'un tube qu'il n'utilise pas, le `read()` côté réception ne recevra jamais EOF et bloquera indéfiniment. Chaque processus i ne conserve que deux descripteurs :

- `tubes[i][0]` — lecture depuis son tube entrant
- `tubes[(i+1)%n][1]` — écriture vers le tube sortant

2.3 Utilitaires de synchronisation

Deux fonctions auxiliaires garantissent la propreté entre les étapes :

`vider_tube(fd)` — place le descripteur en mode `O_NONBLOCK`, consomme tous les entiers résiduels, puis restaure le mode bloquant. Appelé par tous les processus après la partie c.

`barriere()` — un jeton fait exactement un tour de l’anneau : P0 l’envoie et attend son retour ; les autres le reçoivent et le retransmettent. Garantit que tous les processus ont terminé l’étape courante avant de passer à la suivante.

Listing 2 – Barrière de synchronisation

```
1 void barriere(int rang, int n, int lecture, int ecriture) {
2     int jeton = 1;
3     if (rang == 0) {
4         write(ecriture, &jeton, sizeof(int));
5         read(lecture, &jeton, sizeof(int));
6     } else {
7         read(lecture, &jeton, sizeof(int));
8         write(ecriture, &jeton, sizeof(int));
9     }
10 }
```

3 Partie b — Comptage des processus

3.1 Principe

P0 initialise un compteur à 1 et l’injecte dans l’anneau. Chaque processus P_i ($i > 0$) reçoit le compteur, l’incrémente de 1 et le retransmet. Quand le compteur revient à P0, il vaut exactement n .

Listing 3 – Comptage en anneau — `compter_processus()`

```
1 void compter_processus(int rang, int n, int lecture, int ecriture)
2 {
3     int compteur;
4     if (rang == 0) {
5         compteur = 1;
6         write(ecriture, &compteur, sizeof(int));
7         read(lecture, &compteur, sizeof(int));
8         printf("[Partie_b] Nombre de processus dans l'anneau: %d\n", compteur);
9     } else {
10        read(lecture, &compteur, sizeof(int));
11        compteur++;
12        write(ecriture, &compteur, sizeof(int));
13    }
```

3.2 Résultats

n passé en argument	Résultat affiché
2	Nombre de processus dans l'anneau : 2
3	Nombre de processus dans l'anneau : 3
5	Nombre de processus dans l'anneau : 5
20	Nombre de processus dans l'anneau : 20

Le résultat est correct pour toutes les valeurs de $n \in [2, 20]$.

4 Partie c — Circulation d'un message

4.1 Principe

P0 initialise `message = 10` et l'envoie dans l'anneau. Les autres processus le reçoivent et le retransmettent sans modification. Quand le message revient à P0, il le décrémente et envoie la nouvelle valeur. Ce cycle se répète jusqu'à ce que `message` atteigne 0. P0 envoie alors le 0 final — signal d'arrêt pour tous.

Listing 4 – Circulation du message — `circuler_message()`

```
1 void circuler_message(int rang, int n, int lecture, int ecriture) {
2     int message;
3     if (rang == 0) {
4         message = 10;
5         printf("[Partie_c]_P0_initialise_le_message_a_%d\n",
6             message);
7         while (message > 0) {
8             write(ecriture, &message, sizeof(int));
9             read(lecture, &message, sizeof(int));
10            printf("[Partie_c]_P0_recoit:_%d->decrement->_%d\n",
11                , message, message - 1);
12            message--;
13        }
14        write(ecriture, &message, sizeof(int)); /* envoie 0 =
15            signal fin */
16        printf("[Partie_c]_Message_=0,_arret_de_l'anneau\n");
17        read(lecture, &message, sizeof(int)); /* attend retour du
18            0 */
19    } else {
20        while (1) {
21            read(lecture, &message, sizeof(int));
22            write(ecriture, &message, sizeof(int));
23            if (message == 0) break;
24        }
25    }
26 }
```

4.2 Point critique — retour du 0 final

Bug initial : tube pollué entre les parties c et d

Dans la version initiale, P0 envoyait le 0 final et **sortait immédiatement** de la fonction sans attendre son retour. Ce 0 faisait le tour de l'anneau ($P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_0$) et revenait dans le tube d'entrée de P0. Mais P0 appelait déjà `vider_tube()` à ce moment, qui consommait silencieusement ce 0. La barrière suivante envoyait alors son jeton prématurément, qui arrivait dans `circuler_pids` comme première valeur de données — d'où des sommes de PIDs aberrantes (1427058718836978 pour $n = 3$ au lieu de $\approx 508\,086$).

Correction : `read()` du 0 en retour avant de quitter

Un simple `read(lecture, &message, sizeof(int))` ajouté après l'envoi du 0 final garantit que P0 attend son propre signal de retour, vidant proprement son tube **avant** l'appel à `vider_tube()` dans `main.c`.

5 Partie d — Circulation des PIDs et somme

5.1 Principe

P0 envoie son propre PID dans l'anneau. Chaque processus P_i reçoit la somme partielle, y ajoute son PID, et la retransmet. Quand la somme revient à P0, elle contient $\sum_{i=0}^{n-1} \text{PID}(P_i)$.

Après l'affichage, P0 envoie un signal de fin -1 qui fait un tour complet pour débloquent tous les autres processus.

Listing 5 – Circulation des PIDs — `circuler_pids()`

```
1 void circuler_pids(int rang, int n, int lecture, int ecriture) {
2     int somme = 0;
3     pid_t mon_pid = getpid();
4
5     if (rang == 0) {
6         somme = (int)mon_pid;
7         printf("[Partie_d]_P0_PID=_%d\n", (int)mon_pid);
8         write(ecriture, &somme, sizeof(int));
9         read(lecture, &somme, sizeof(int));
10        printf("[Partie_d]_Somme_finale_des_PIDs:_%d\n", somme);
11        int fin = -1;
12        write(ecriture, &fin, sizeof(int)); /* signal de fin */
13        read(lecture, &fin, sizeof(int)); /* attend retour */
14    } else {
15        read(lecture, &somme, sizeof(int));
16        somme += (int)mon_pid;
17        write(ecriture, &somme, sizeof(int));
18        printf("[Partie_d]_P%d_PID=_%d\n", rang, (int)mon_pid);
19        int fin;
20        read(lecture, &fin, sizeof(int));
```



```

21     write(ecriture, &fin, sizeof(int));
22 }
23 }

```

5.2 Vérification des résultats

Les PIDs sont attribués de façon consécutive par le noyau ($P_0, P_0+1, \dots, P_0+n-1$).

La somme théorique vaut donc : $S = n \cdot \text{PID}(P_0) + \frac{n(n-1)}{2}$

n	PID(P0)	Somme théorique	Somme affichée	Correct ?
2	169 359	$2 \times 169\,359 + 1 = 338\,719$	338 719	Oui
3	169 361	$3 \times 169\,361 + 3 = 508\,086$	508 086	Oui
4	169 364	$4 \times 169\,364 + 6 = 677\,462$	677 462	Oui
5	169 368	$5 \times 169\,368 + 10 = 846\,850$	846 850	Oui
8	169 408	$8 \times 169\,408 + 28 = 1\,355\,292$	1 355 292	Oui
20	169 417	$20 \times 169\,417 + 190 = 3\,388\,530$	3 388 530	Oui

Toutes les sommes sont exactes. La formule $n \cdot \text{PID}_0 + n(n-1)/2$ est vérifiée pour chaque valeur de n testée.

6 Partie e — Élection (algorithme de Chang-Roberts)

6.1 Principe de l'algorithme

L'algorithme de Chang-Roberts résout le problème d'élection dans un anneau unidirectionnel asynchrone. Chaque processus envoie son identifiant (PID) dans l'anneau et applique les règles suivantes à la réception d'un candidat :

Candidat reçu	Action
> mon PID	Transmettre au voisin suivant
= mon PID	Je suis élu — envoyer -1 (signal de fin)
< mon PID	Absorber (ne rien transmettre)
Reçu -1	Transmettre (si pas élu) puis se terminer

Seul le processus ayant le **plus grand PID** verra son identifiant revenir intact après un tour complet : c'est le seul à ne jamais être absorbé.

Listing 6 – Élection de Chang-Roberts — `election()`

```

1 void election(int rang, int n, int lecture, int ecriture) {
2     pid_t mon_pid = getpid();
3     int candidat, élu = 0;
4
5     candidat = (int)mon_pid;
6     write(ecriture, &candidat, sizeof(int)); /* injection initiale
7     */

```

```

8   while (1) {
9       read(lecture, &candidat, sizeof(int));
10
11      if (candidat == -1) {                /* signal de fin */
12          if (!elu) write(ecriture, &candidat, sizeof(int));
13          break;
14      }
15      if (candidat > (int)mon_pid) {
16          write(ecriture, &candidat, sizeof(int)); /* transmet
17          */
18      } else if (candidat == (int)mon_pid) {
19          printf("[Partie_e]_P%d_(PID=%d)_est_elu_chef_!\n",
20                  rang, (int)mon_pid);
21          elu = 1;
22          int fin = -1;
23          write(ecriture, &fin, sizeof(int));      /* envoie fin
24          */
25      }
26      /* candidat < mon_pid : absorbe, ne transmet pas */
27  }

```

6.2 Terminaison

La terminaison est assurée par la propagation du signal -1 :

1. L'élue reçoit son propre PID, se proclame chef et envoie -1.
2. Chaque processus non-élue reçoit -1, le retransmet et sort de la boucle.
3. L'élue reçoit en dernier son propre -1 et sort également.

6.3 Résultats

n	PID max	Processus élu	Correct ?
2	169 360	P1	Oui
3	169 363	P2	Oui
4	169 367	P3	Oui
5	169 372	P4	Oui
6	169 378	P5	Oui
7	169 385	P6	Oui
8	169 415	P7	Oui
20	169 436	P19	Oui

Le processus élu est systématiquement celui ayant le PID le plus élevé, ce qui est cohérent avec l'ordre d'attribution des PIDs par le noyau Linux lors des `fork()` successifs.

7 Analyse du bug principal et correction

7.1 Symptôme observé

Lors des premiers tests, la partie d produisait des sommes aberrantes pour $n \geq 3$:

n	Somme affichée (bugué)	Somme correcte
3	1 427 058 718 836 978	508 086
4	2 140 633 175 329 013	677 462
5	2 140 697 600 004 595	846 850

7.2 Cause racine

La séquence d'événements dans `circuler_message` côté P0 était :

1. P0 envoie le 0 final dans l'anneau.
2. P0 **sort immédiatement** de la fonction.
3. P0 appelle `vider_tube(lecture)` en mode `O_NONBLOCK`.
4. Le 0 est encore en transit ($P_1 \rightarrow P_2 \rightarrow \dots \rightarrow P_0$).
5. Quand il revient dans le tube de P0, `vider_tube` le consomme silencieusement.
6. La `barriere()` envoie immédiatement son jeton dans l'anneau.
7. Ce jeton arrive à P1 **comme si c'était la première valeur de** `circuler_pids`.
8. P1 lit `jeton = 1` au lieu de la somme partielle $\rightarrow$ résultat faussé.

7.3 Correction appliquée

Un `read()` du 0 de retour avant de quitter `circuler_message` :

Listing 7 – Correction — attente du retour du 0 final

```
1 /* Envoie le 0 final */
2 write(ecriture, &message, sizeof(int));
3 printf("[Partie_c] Message=0, arret de l'anneau\n");
4
5 /* CORRECTION : attend que le 0 ait fait le tour complet */
6 /* Garantit que le tube de P0 est propre avant vider_tube() */
7 read(lecture, &message, sizeof(int));
```

Leçon retenue

En programmation concurrente avec pipes, un processus ne doit jamais supposer que le tube est vide sans s'être synchronisé avec tous ses voisins. Ici, `vider_tube` était appelé trop tôt, avant que le 0 final ait complété son transit dans l'anneau. La règle générale : **chaque donnée injectée doit être consommée avant de changer de phase.**

8 Synthèse et conclusion

8.1 Bilan des résultats

Partie	Fonctionnalité	Résultat
a	Création de l'anneau et fermeture des descripteurs	Correct
b	Comptage des processus par circulation d'un jeton	Correct
c	Circulation du message ($10 \rightarrow 0$)	Correct
d	Somme des PIDs (vérifiée analytiquement)	Correct
e	Élection Chang-Roberts (PID max élu)	Correct

8.2 Conclusion

Ce projet a permis de mettre en pratique les mécanismes fondamentaux de la programmation système Unix : création de processus par `fork()`, communication inter-processus par `pipe()`, gestion rigoureuse des descripteurs de fichiers, et synchronisation entre processus parallèles.

L'algorithme de Chang-Roberts illustre comment un problème distribué (élection d'un coordinateur) peut être résolu de manière élégante avec seulement $O(n \log n)$ messages dans le cas moyen, uniquement par communication locale entre voisins immédiats.

Le débogage du bug de synchronisation entre les parties c et d a renforcé un principe essentiel : **en concurrence, toute donnée injectée dans un canal de communication doit être explicitement consommée avant de changer de phase**, sous peine de corruption silencieuse des données de la phase suivante.

Références

- **Sujet** : M. Noufel & M. ElAmrani, *Programmation Système — Projet N°2 : Processus en Anneau*, UIR ESIN, 2025-2026
- **Cours** : M. Noufel & M. ElAmrani, *TP0 à TP5 — Processus, Tubes, Signaux et Synchronisation*, UIR ESIN, 2025-2026
- **Cours** : Prof. Eric GASCARD & Prof. Nicolas PALIX, *C Programmation Système (CPS)*, S5, Polytech Grenoble INP, 2024-2025
- **Cours** : Prof. Michael PERIN, *Programmation C (PROG C)*, S5, Polytech Grenoble INP, 2024-2025
- **Algorithme** : E. Chang, R. Roberts, *An Improved Algorithm for Decentralized Extrema-Finding in Circular Configurations of Processes*, Communications of the ACM, 22(5), 1979
- **Ouvrage** : W. Richard Stevens, Stephen A. Rago, *Advanced Programming in the UNIX Environment*, 3^e édition, Addison-Wesley, 2013
- **Assistant IA** : Claude (Sonnet 4.6), Anthropic, <https://claude.ai>